

Тема 2. Класове и обекти

Клас – шаблон, който описва същността на реален обект или софтуерен обект, описвайки атрибути (свойства) и методи (поведение) на обектите. Описва съвкупност от променливи (нар. полета) от различни типове, в това число и обекти на други класове, както и методи, предназначени да работят с тези полета. Т.е. в описанието на класа се съдържа информацията за това какви полета и методи имат обектите, създавани въз основата на дадения клас.

Обект – екземпляр на класа.

Метод – Код, предназначен за работа с полетата на обекта и не само. Метод на обекта е именуван блок с команди, които се изпълняват при извикването на метода.

Дефиниране на клас

```
class име_на_класа{  
    // описание на класа – полета и методи  
}
```

```
class MyClass{  
    // Клас само с полета  
    int number;  
    double value;  
}
```

При създаването на обект за всяко от неговите полета се заделя място в паметта, така че всеки обект си има своите полета („персонални“)!

Създаване на обект

- деклариране на обектна променлива, чрез която ще се осъществява достъп до обекта;
- самото създаване на обекта и „свързването“ му с обектна променлива;

```
име_на_класа обектна_променлива;  
обектна_променлива = new име_на_класа();  
или
```

```
име_на_класа обектна_променлива = new име_на_класа();
```

*Декларирането на обектна променлива не означава създаване на обект. Т.е. променливата съществува, за нея се заделя място в паметта, но това не е обект. Обектът се създава с помощта на оператора **new**, след който се задава*

Платформено независими програмни езици

Семинарно упражнение 2

името на класа, на основата на който се създава обекта, и кръгли скоби. На практика след операцията **new** се указва конструкторът за създаване на обекта. Това е специален метод, който се извиква при създаването на обекта. Името на конструкторът съвпада с името на класа. В кръглите скоби се подават аргументите на конструктора (ако са необходими). Ако конструкторът няма аргументи, кръглите скоби са празни.

Разликата между обектна променлива и обект: стойността на обектната променлива се явява „адрес“ на обекта или референция към обекта. Т.е обектната променлива „знае“ къде в паметта се съхранява обекта и има достъп до него.

```
Myclass object;  
object = new Myclass();  
или  
Myclass object = new Myclass();
```

Пример 1: Използване на обекти: да се създаде клас с три полета (две целочислени и едно от тип double, в което да се запише средно-аритметичната стойност на другите две полета). Да се създаде обект в главния клас и да се присвоят стойностите на полетата. Да се изведе резултатът в конзолата.

```
class Myclass {  
    int number1;  
    int number2;  
    double value;  
}  
  
public class Primer1 {  
    public static void main(String[] args) {  
        Myclass obj = new Myclass();  
        obj.number1 = 6;  
        obj.number2 = 7;  
        obj.value = (obj.number1 + obj.number2)/2.0;  
        System.out.println(obj.value);  
    }  
}
```

Класове с методи

При дефиниране на метода се указва типът на връщания от метода резултат (определя се от идентификатор на типа), името на метода и списък с аргументи. Всичко изброено до тук се нарича сигнатура. Списъкът с аргументи се поставя в кръгли скоби след името на метода като аргументите се разделят

Платформено независими програмни езици
Семинарно упражнение 2

със запетая. За всеки аргумент се указва тип и име. Тялото на метода се поставя във фигурни скоби. Името на метода избираме сами.

```
тип_на_резултата име_на_метода(аргументи){  
    // тяло на метода  
}
```

Ако методът не връща стойност, се използва ключовата дума **void**. Ако връща стойност, в тялото тя се указва след инструкцията **return**. Методите се обръщат към полетата на същия обект, от който са извикани. При обръщение към поле или метод на обект първо се указва името на обекта, а след това чрез точка се поставя името на полето или метода (с кръгли скоби и ако е необходимо, с аргументи). По този начин могат да се присвояват стойности на полетата или да се подават стойности на аргументите на метода.

Пример 2: Използване на класове с методи: да се създаде клас с две полета (едно целочислено и едно от тип `double`). Да се създадат два метода, които не връщат резултат: чрез първият метод да се изчисли корен квадратен на първото поле, чрез вторият метод да се принтират в конзолата двете числа. Да се създаде обект в главния клас и да се присвои стойността на първото поле. Да се изведе резултатът в конзолата.

```
import java.lang.Math;  
  
class Myclass1 {  
    int number;  
    double value;  
    void set(int num){  
        number = num;  
        value = Math.sqrt(num);  
    }  
    void show(){  
        System.out.print(number + ": " + value);  
    }  
}  
  
public class Primer2 {  
  
    public static void main(String[] args) {  
        Myclass1 A = new Myclass1();  
        A.set(15);  
        A.show();  
    }  
}
```

Пример 3: Да се модифицира класа дефиниран в предходния пример, като първият метод връща резултат корен квадратен на неговия аргумент.

```
import java.lang.Math;

class Myclass1 {
    int number;
    double value;
    double set(int num){
        number = num;
        value = Math.sqrt(num);
        return value;
    }
    void show(){
        System.out.print(number + ": ");
    }
}

public class Primer3 {

    public static void main(String[] args) {
        Myclass1 A = new Myclass1();
        double val = A.set(16);
        A.show();
        System.out.println(val);
    }
}
```

Методи и конструктори

В Java в един и същи клас могат да бъдат дефинирани няколко метода с едно и също име. Тези методи ще се различават по типа и/или броя на аргументите им. Това се нарича презареждане на методи. Начинът, по който се извиква метода, определя кой точно метод ще се извика.

Пример 4:

```
class Myclass3 {
    int number;
    char symbol;
    double value;
    void set(int n) {
        number = n;
    }
    void set(char s) {
        symbol = s;
    }
}
```

Платформено независими програмни езици
Семинарно упражнение 2

```
void set(double v) {
    value = v;
}
void set(int n, char s, double v) {
    set(n);
    set(s);
    set(v);
}
void set() {
    set(1, '1', 1.00);
}
void show() {
    System.out.println("Стойностите на полетата са " + number + " " +
symbol + " " + value);
}
void show(String text) {
    System.out.println(text + " " + number + " " + symbol + " " + value);
}
void show(String text1, String text2, String text3) {
    System.out.println(text1 + " " + number);
    System.out.println(text2 + " " + symbol);
    System.out.println(text3 + " " + value);
}
}

public class Primer4 {
    public static void main(String[] args) {
        Myclass3 obj1 = new Myclass3();
        Myclass3 obj2 = new Myclass3();
        Myclass3 obj3 = new Myclass3();
        obj1.set(10);
        obj1.set('B');
        obj1.set(123.45);
        System.out.println("Object 1:");
        obj1.show();
        obj2.set(100, 'S', 987.65);
        System.out.println("Object 2:");
        obj2.show();
        obj3.set();
        System.out.println("Object 3:");
        obj3.show();
        obj3.set(300, 'R', 34.56);
        obj3.show("New values are:");
    }
}
```

Платформено независими програмни езици
Семинарно упражнение 2

```
obj3.show("Числото е:", "Char променливата е:", "Double  
променливата е:");  
}  
}
```

Изход в конзолата:

Object 1:

Стойностите на полетата са 10 B 123.45

Object 2:

Стойностите на полетата са 100 S 987.65

Object 3:

Стойностите на полетата са 1 1 1.0

New values are: 300 R 34.56

Числото е: 300

Char променливата е: R

Double променливата е: 34.56

Конструктор се нарича методът, който се извиква автоматично при създаването на обект на класа. Чрез него се определят допълнителни действия, които трябва да бъдат изпълнени при създаването на обекта. Особености:

- името на конструктора съвпада с името на класа;
- конструкторът не връща резултат, но и ключовата дума **void** не се използва;
- конструкторът може да има аргументи и да бъде презареждан, т.е. в един клас може да има повече от един конструктор.

При наличие на конструктори обектът се създава както следва:

име_на_класа променлива = new конструктор(аргументи);

Пример 5:

```
class Myclass4 {  
    int number;  
    char symbol;  
    double value;  
    Myclass4(int n) {  
        number = n;  
    }  
    Myclass4(char s) {  
        symbol = s;  
    }  
    Myclass4(double v) {  
        value = v;  
    }  
    Myclass4(int n, char s, double v) {  
        number = n;  
    }  
}
```

Платформено независими програмни езици
Семинарно упражнение 2

```
        symbol = s;
        value = v;
    }
    Myclass4() {
        number = 1;
        symbol = '1';
        value = 1.00;
    }
    void show() {
        System.out.println("Стойностите на полетата са " + number + " " +
symbol + " " + value);
    }
    void show(String text) {
        System.out.println(text + " " + number + " " + symbol + " " + value);
    }
    void show(String text1, String text2, String text3) {
        System.out.println(text1 + " " + number);
        System.out.println(text2 + " " + symbol);
        System.out.println(text3 + " " + value);
    }
}

public class Primer5 {
    public static void main(String[] args) {
        Myclass4 obj1 = new Myclass4(10);
        Myclass4 obj2 = new Myclass4('B');
        Myclass4 obj3 = new Myclass4(123.45);
        System.out.println("Конструктор с един аргумент:");
        System.out.println("Стойността е " + obj1.number);
        System.out.println("Стойността е " + obj2.symbol);
        System.out.println("Стойността е " + obj3.value);
        System.out.println("Конструктор с три аргумента:");
        Myclass4 obj4 = new Myclass4(300,'R',34.56);
        obj4.show();
        System.out.println("Конструктор без аргумент:");
        Myclass4 obj5 = new Myclass4();
        obj5.show("Новите стойности са ");
        System.out.println("или може и така за конструктор без аргументи");
        obj5.show("Числото е:", "Char променливата е:", "Double
променливата е:");
    }
}
```

Изход в конзолата:

Платформено независими програмни езици

Семинарно упражнение 2

Конструктор с един аргумент:
Стойността е 10
Стойността е B
Стойността е 123.45
Конструктор с три аргумента:
Стойностите на полетата са 300 R 34.56
Конструктор без аргумент:
Новите стойности са 1 1 1.0
или може и така за конструктор без аргументи
Числото е: 1
Char променливата е: 1
Double променливата е: 1.0

Статични полета и методи

Статичните полета са общи за всички обекти на даден клас. Статичните полета съществуват независимо от това дали са били създадени обекти на конкретния клас или не.

Статичните методи (аналогично на статичните полета) не са обвързани с обектите и съществуват сами по себе си.

Обръщането към статично поле или статичен метод се прави чрез класа!

Пример 6:

```
class Primer6{
    static void show(int[][] nums){
        for(int i=0;i<nums.length;i++){
            for(int j=0;j<nums[i].length;j++){
                System.out.print(nums[i][j]+" ");
            }
            System.out.println("");
        }
    }

    public static void main(String[] args){
        int[][] arr1={{1,2,3},{4,5,6}};
        int[][] arr2={{1},{2,3},{4,5,6},{7,8,9,10}};
        System.out.println("Масив arr1:");
        show(arr1);
        System.out.println("Масив arr2:");
        show(arr2);
    }
}
```


Частни членове на класа

Когато полетата и методите са частни (**private**) достъп до тях има само в рамките на програмния код на класа, т.е. извън класа не може да му бъде присвоена стойност.

Ключова дума this

this – референция към обекта, от който се извиква метода;

Използва се:

- когато при дефиниране на метод в даден клас се налага да се обърнем към обекта, от който е извикан метода. **НО в този момент обектът все още не съществува!**
- когато в тялото на класа от една версия на конструктор се налага да се извика друга.
- правило: в случай на съвпадение на имената на полетата и локалните променливи (аргументите) приоритет имат локалните променливи. За да се обърнем към полето се използва **this**.

Пример 7:

```
class MyClassThis{
    // Поле на класа:
    int num;
    void set(int num){
        // Използване на референцията this:
        this.num=num;
    }
    void show(){
        System.out.println("Числото е: "+num);
    }
}
// Главен клас:
class Primer7{
    public static void main(String[] args){
        // Създаване на обект:
        MyClassThis obj=new MyClassThis();
        // Присвояване на стойност на полето:
        obj.set(5);
        // Показване на стойността на полето:
        obj.show();
    }
}
```

Задача за самостоятелна работа:

Създайте специален клас за работа с квадратна матрица. Да включва следните методи:

1. за създаване на квадратна матрица.
2. за създаване на матрица, чиито стойности за случайни целочислени числа.
3. за създаване на единична матрица.
4. за изчисляване на сумата от елементите от главния диагонал на квадратна матрица.
5. за извеждане на елементите от главния диагонал.
6. за изчисляване на сумата от елементите от обратния диагонал на квадратна матрица.
7. за транспониране на квадратна матрица.
8. за показване на създадените матрици.